

Complete Flow Execution Guide: From Code to Runtime

A Comprehensive Deep Dive into Langbuilder's Component System

Table of Contents

1. [Why Code Needs to Be in String Format](#)
2. [Template Folder Deep Dive](#)
3. [What is FrontendNode](#)
4. [Complete Flow Execution: Step by Step](#)
5. [LangGraph Integration](#)
6. [Example 1: Simple Chat Flow \(Without Tool\)](#)
7. [Example 2: Calculator Tool Flow](#)
8. [Complete Code Execution Trace](#)
9. [When is Each Folder Used? \(UI vs Runtime\)](#)
10. [Detailed Examples: UI Loading vs Runtime Execution](#)
11. [Tracing with Langfuse: Complete Integration](#)

1. Why Code Needs to Be in String Format {#why-code-needs-to-be-in-string-format}

The Core Problem

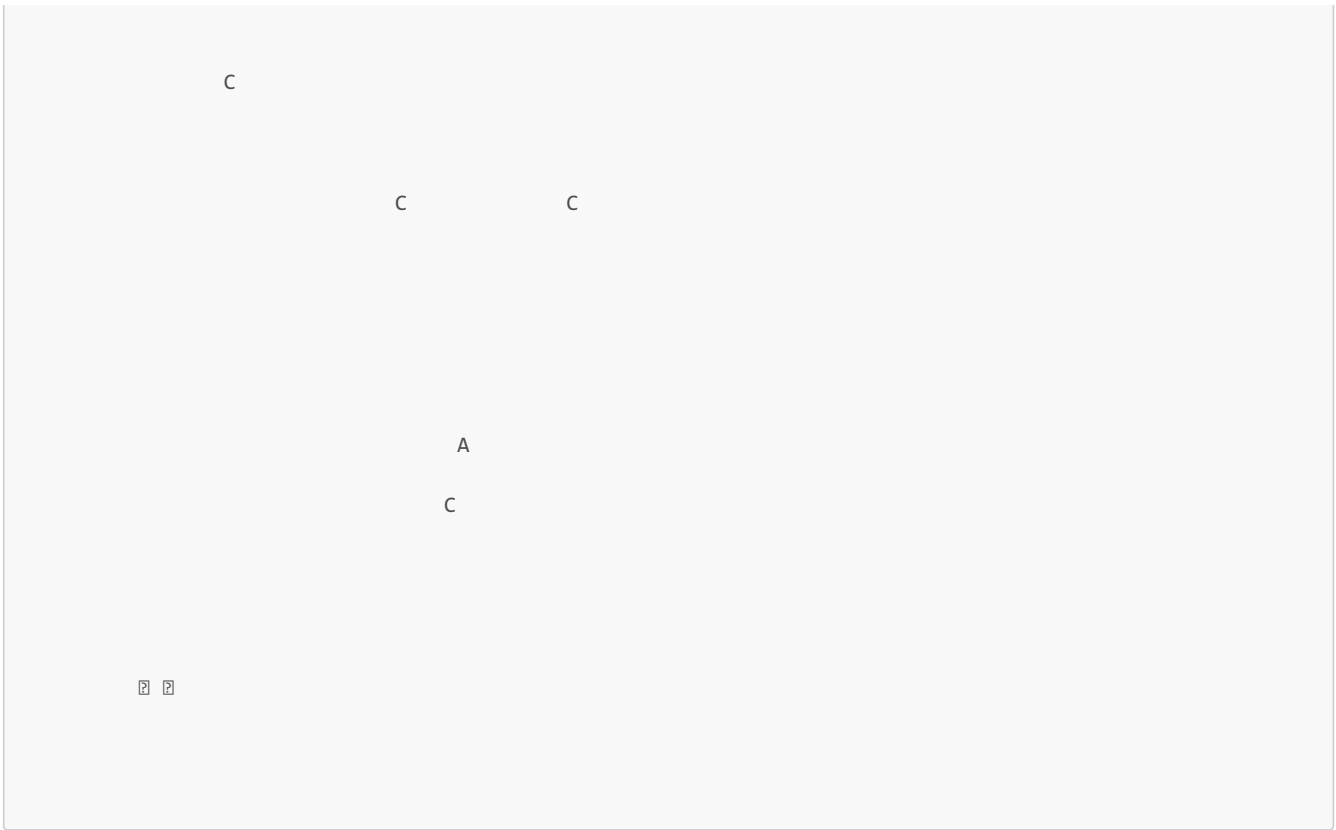
When you create a flow in Langbuilder's visual UI, the system needs to:

1. **Store** the component's Python code in a database (JSON format)
2. **Send** it to the frontend as part of the flow definition
3. **Reconstruct** it at runtime on potentially different servers

Python classes CANNOT be directly stored in JSON or sent over HTTP.

The Solution: Code as String





Key Code Location:

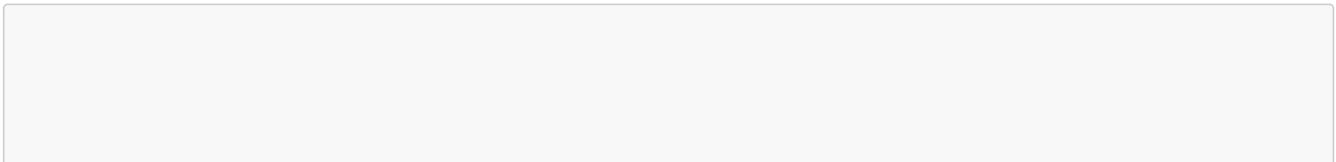


Why This Architecture?

Requirement	Why String Works
Database Storage	JSON can store strings, not Python objects
Network Transfer	HTTP APIs transmit text, not binary classes
Version Control	Flows can be exported/imported as JSON files
User Custom Code	Users can edit component code directly in the UI
Hot Reloading	Changes to code don't require server restart

2. Template Folder Deep Dive {#template-folder-deep-dive}

Folder Structure

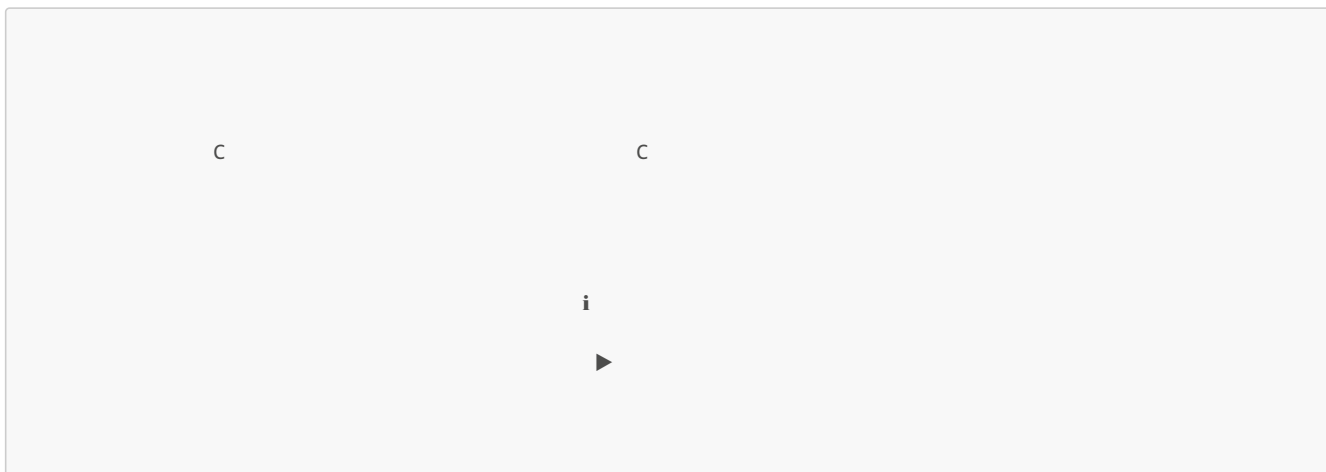


File: - Input/Output Definitions

This file defines HOW inputs and outputs are structured:

File: - Template Configuration

Visual Example: How Template Maps to UI



3. What is FrontendNode {#what-is-frontendnode}

Definition

is the **complete JSON representation** of a component that gets sent to the frontend UI. It contains:

- All metadata (name, icon, description)
- All input fields (with their types, defaults, options)
- All output definitions
- Validation rules
- UI configuration

File:



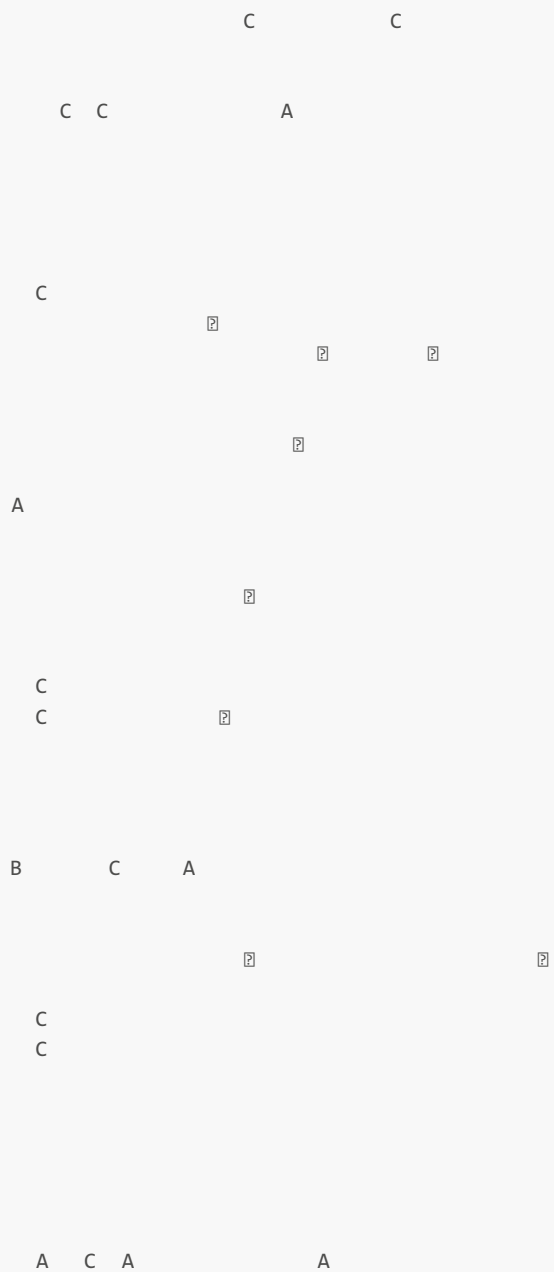
How FrontendNode is Created

C



4. Complete Flow Execution: Step by Step {#complete-flow-execution-step-by-step}

The Big Picture



C

A

B

C

C

?

?

?

?

B

C

?

B

C

C

C

A

A

C

C

?

C

A

A

C

C

C

C

C

C

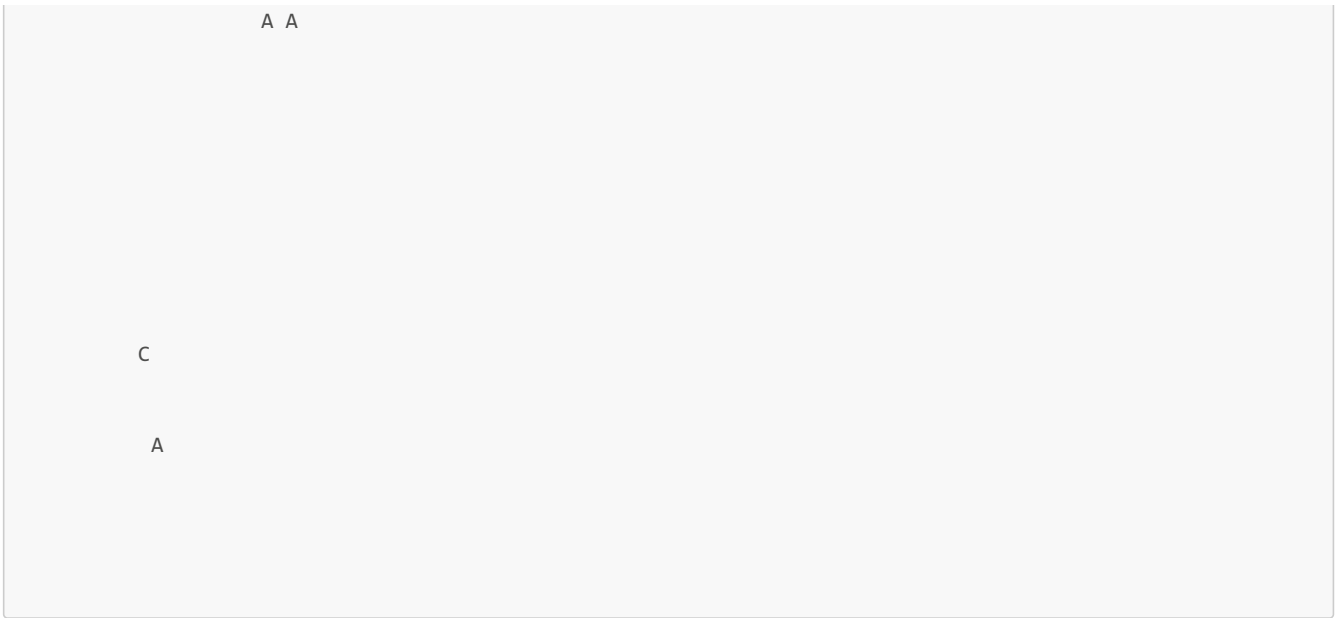
C

C

C

C

C

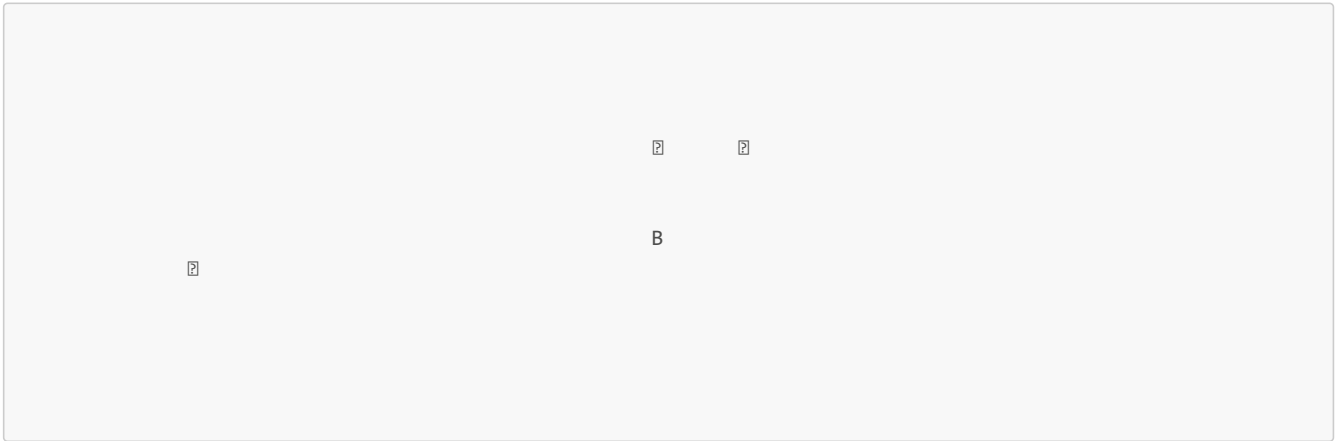


5. LangGraph Integration {#langgraph-integration}

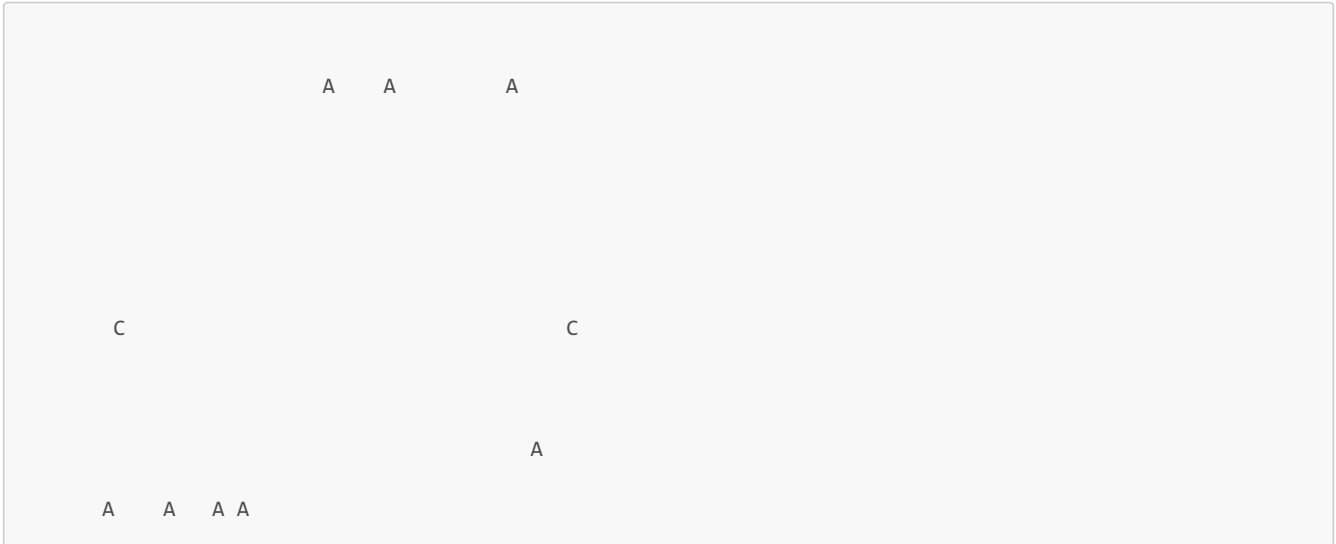
What is LangGraph?

LangGraph is a library from LangChain for building stateful, multi-actor applications with LLMs. Langbuilder uses it as the execution engine.

Folder Structure



How LangGraph Connects to Components



C
B

C

?

?

?

?

A

A

?

?

B

A

?

?

?

C

?

?

C

?

?

A

?

?

?

C

?

?

C

?

?

C

C

?

?

C

?

?

C

?

?

C

?

B

?

?

?

A

A

A

A

C

C

Key File:



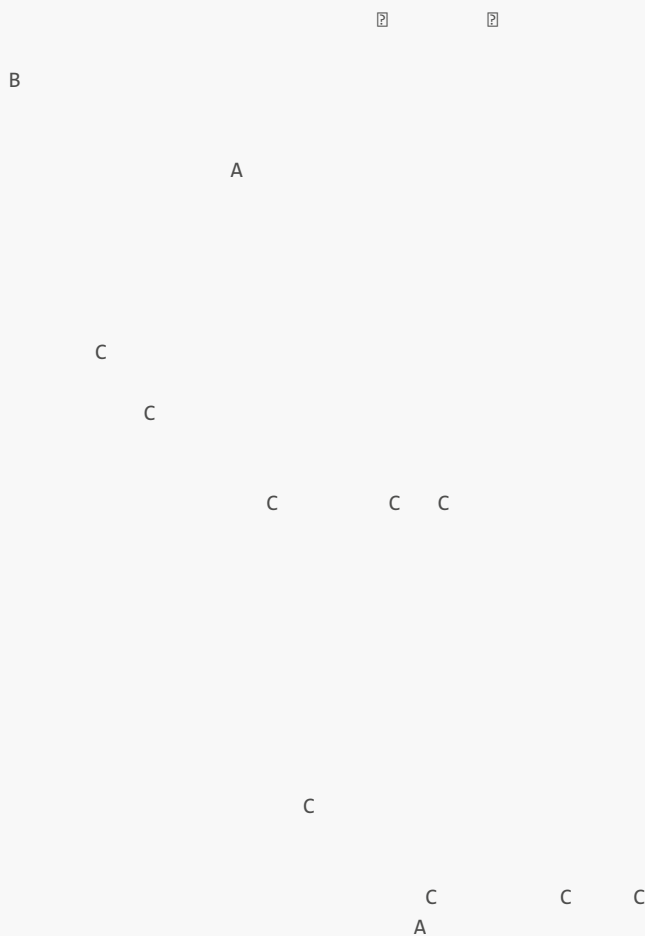
6. Example 1: Simple Chat Flow (Without Tool) {#example-1-simple-chat-flow-without-tool}

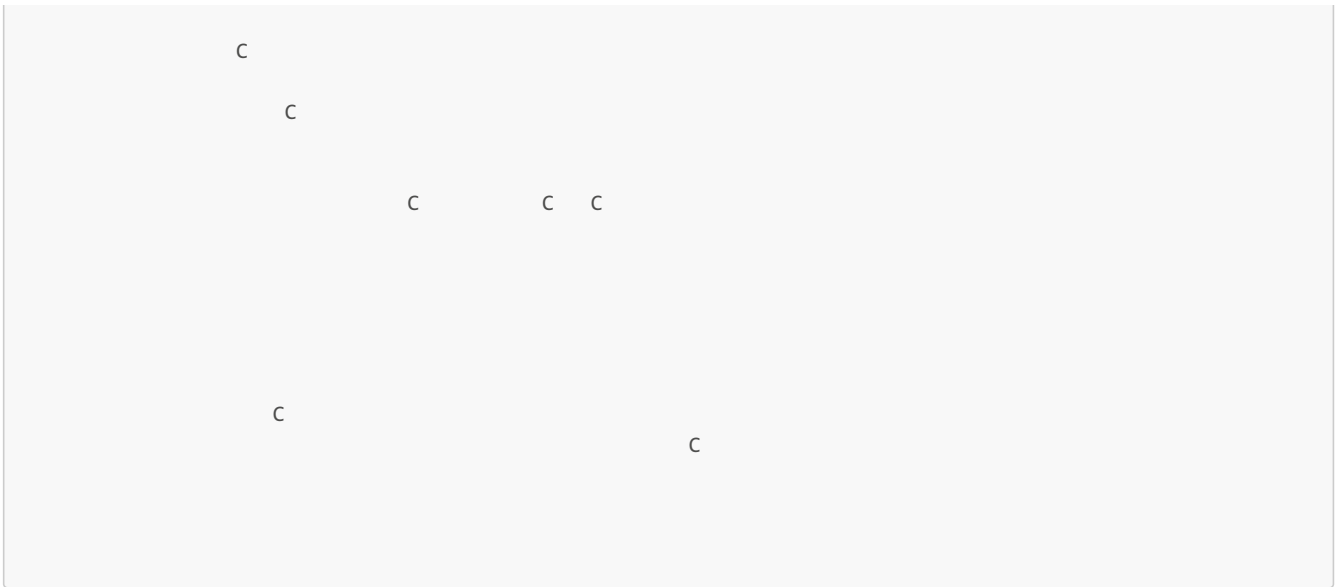
The Flow



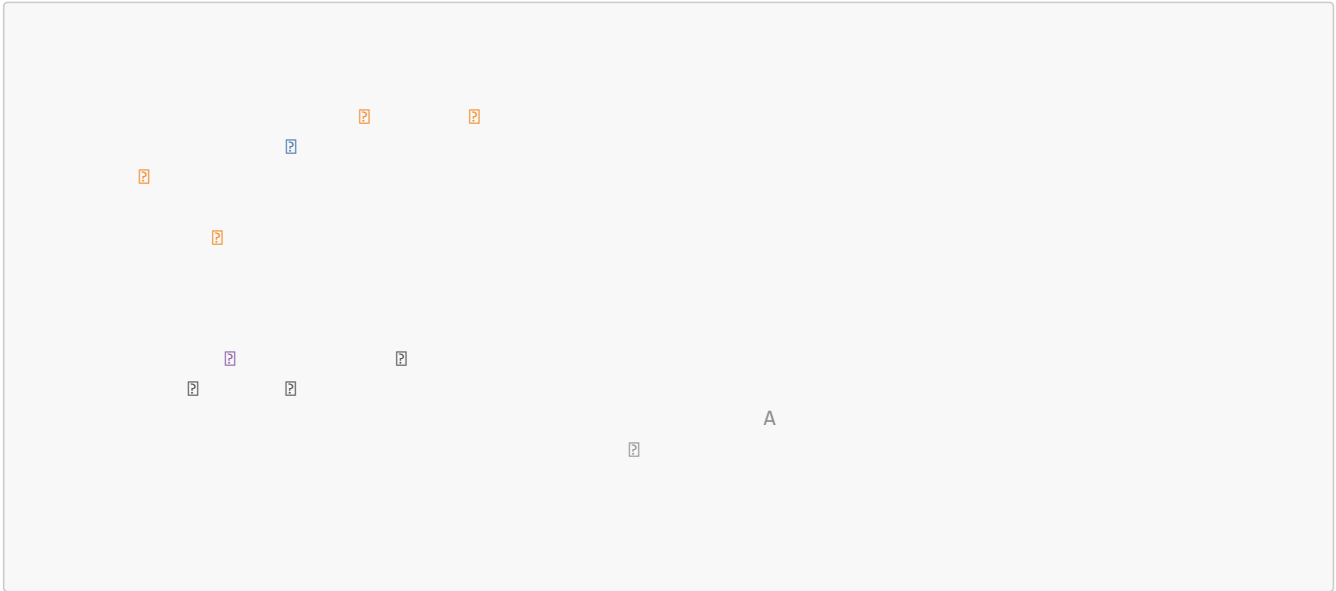
Step-by-Step Execution

STEP 1: User clicks "Run" with input "Hello AI!"

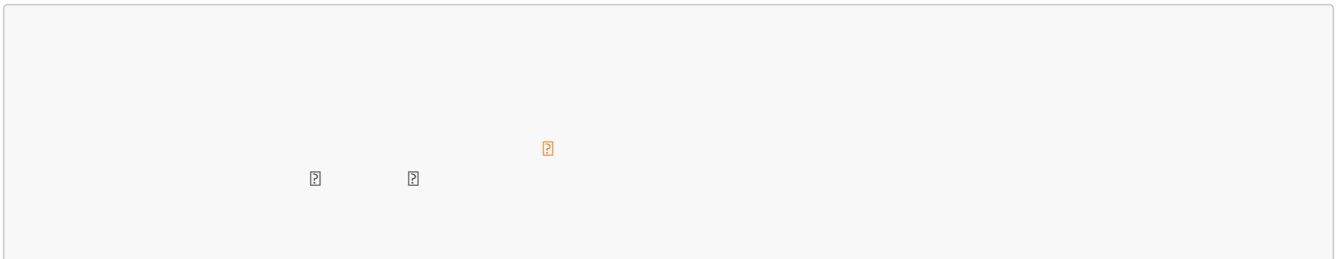
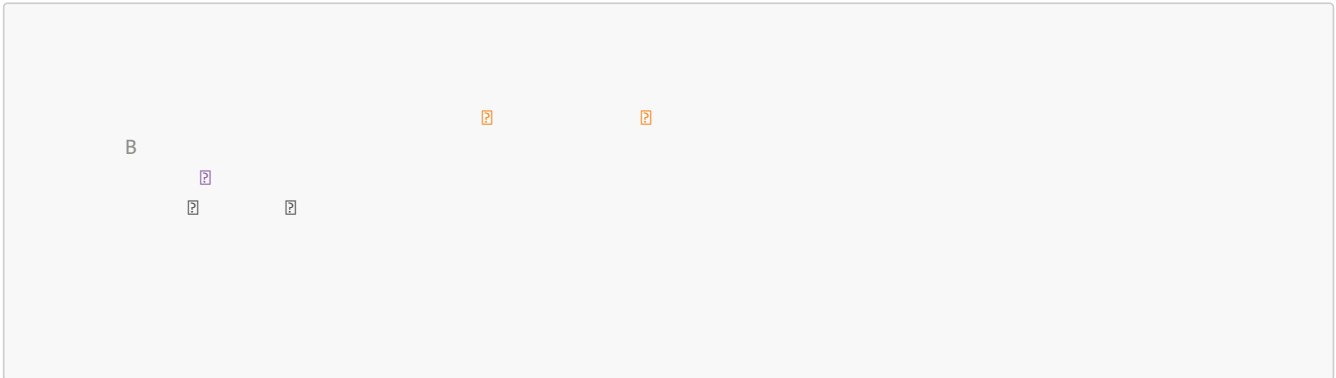


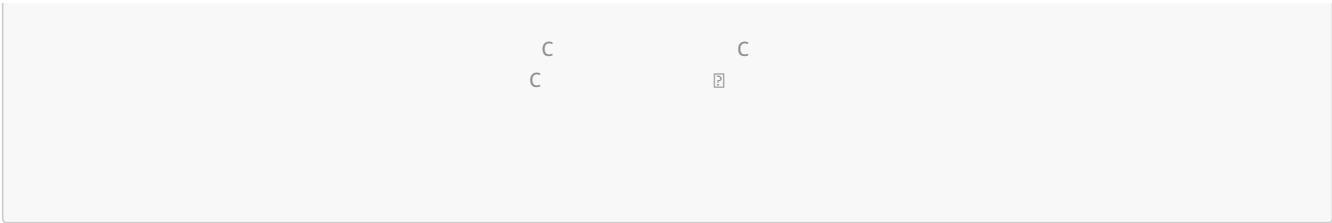


STEP 2: API receives request



STEP 3: Graph is created from payload

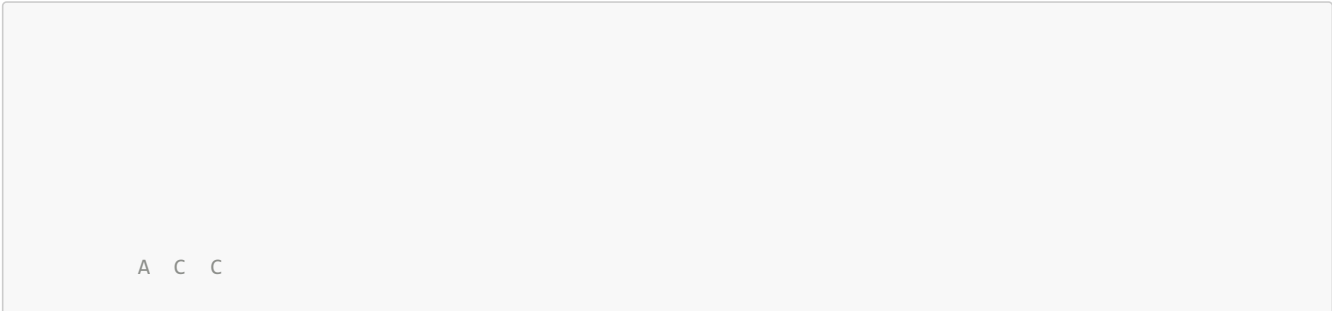
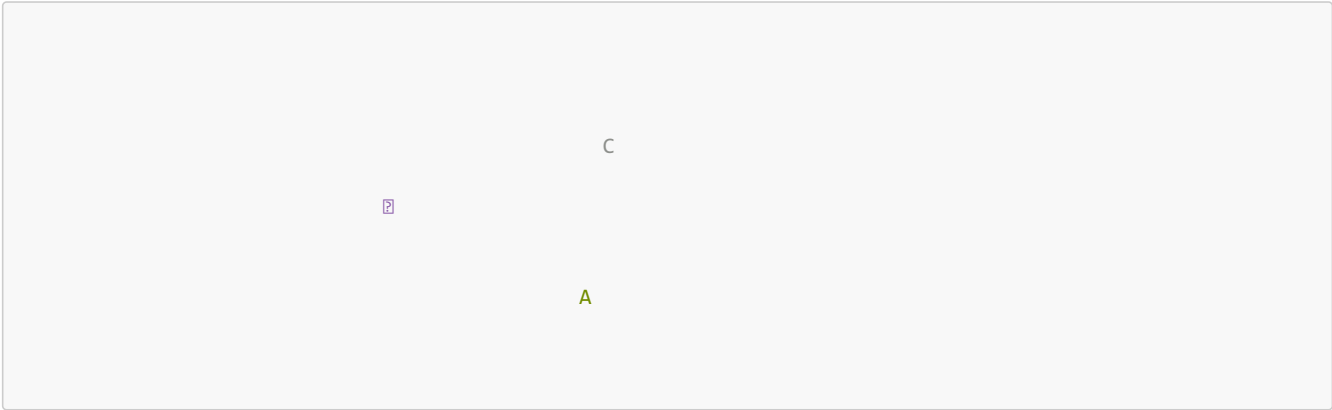


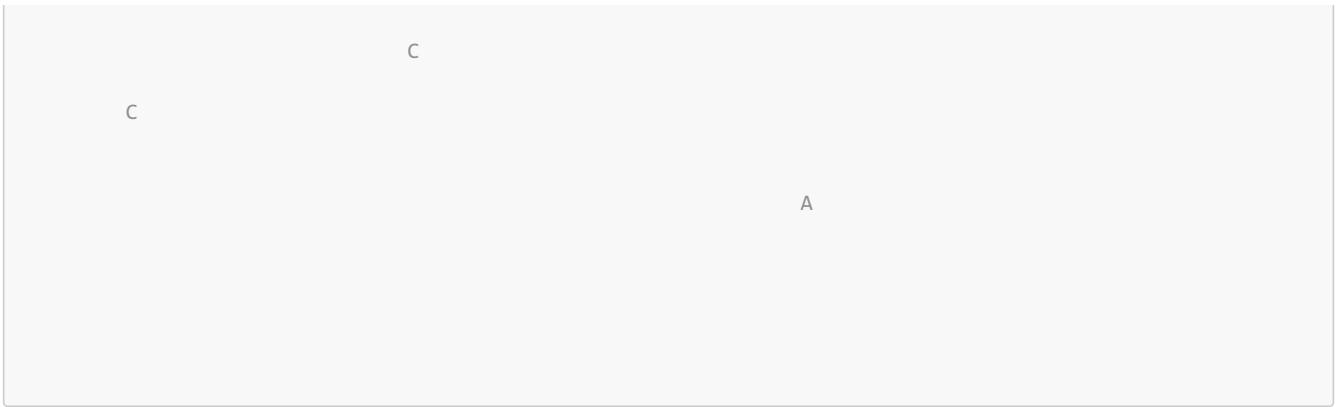


STEP 4: Vertices are sorted by execution order

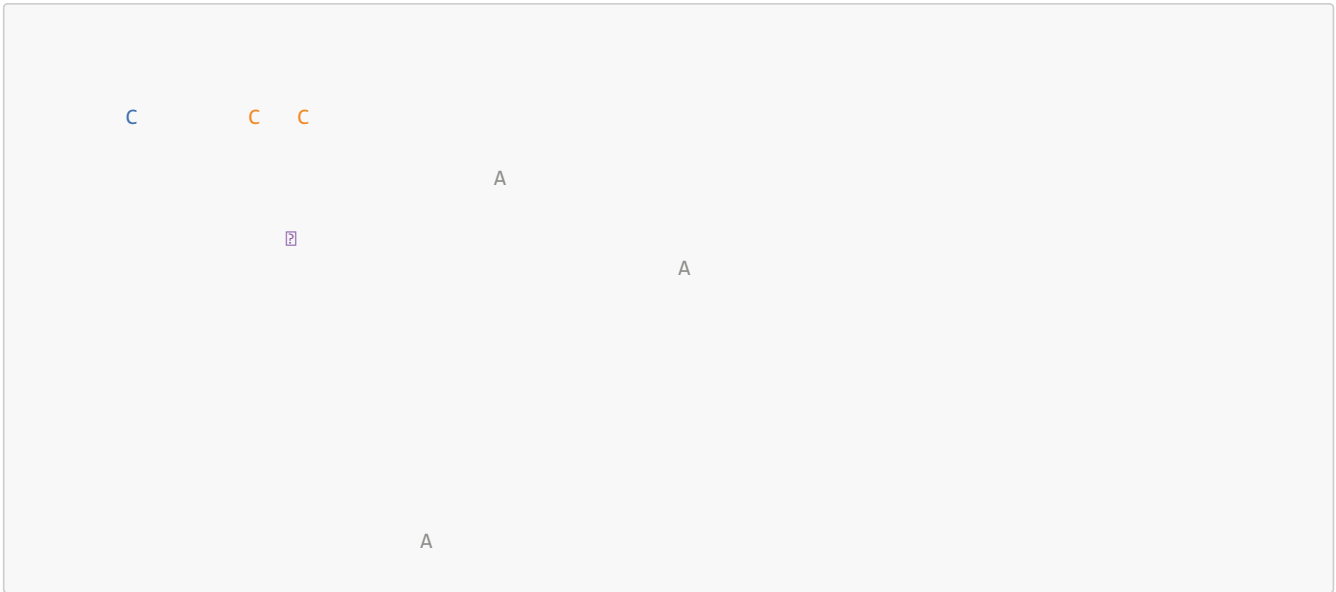


STEP 5: Build ChatInput vertex

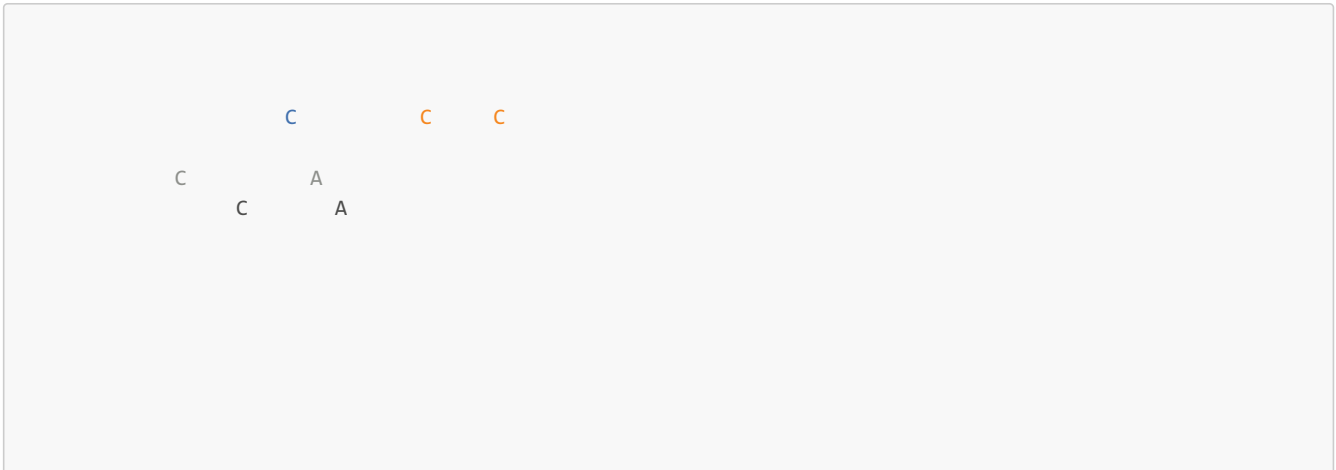
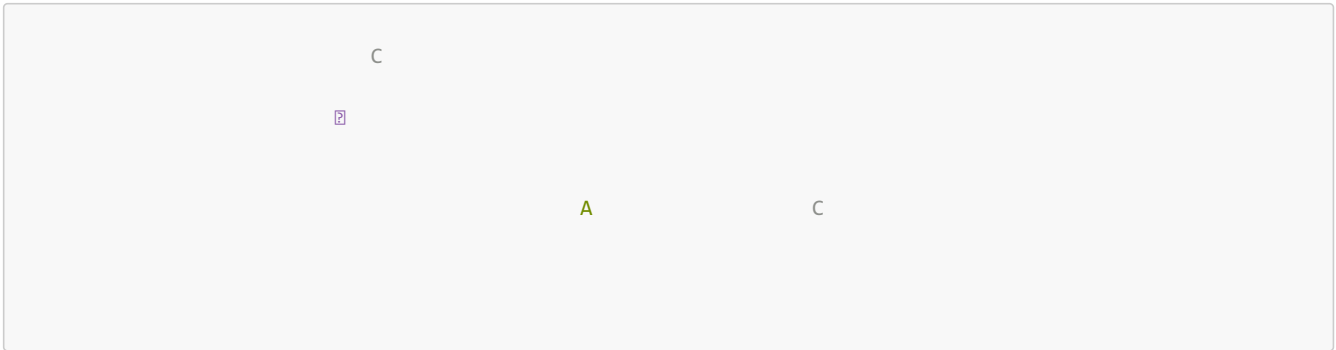


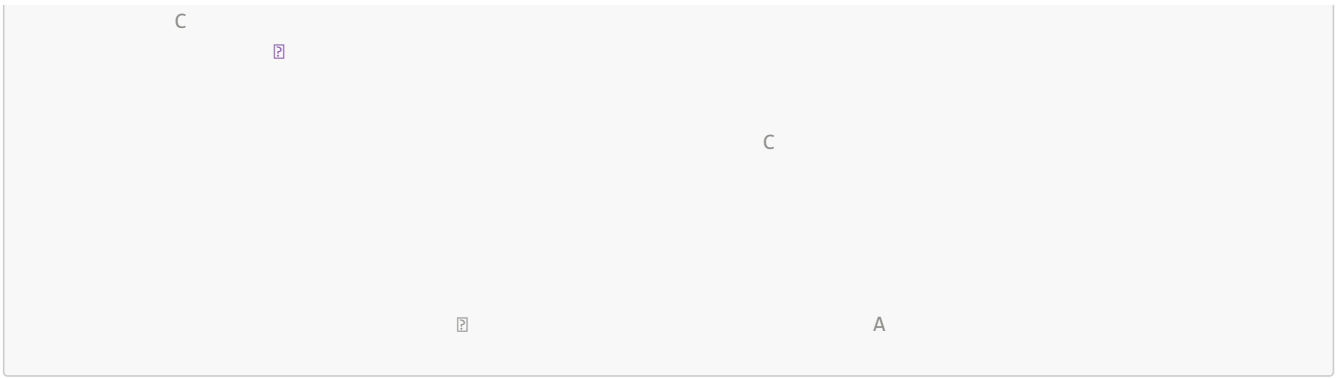


STEP 6: ChatInput.message_response() executes



STEP 7: Build LanguageModel vertex

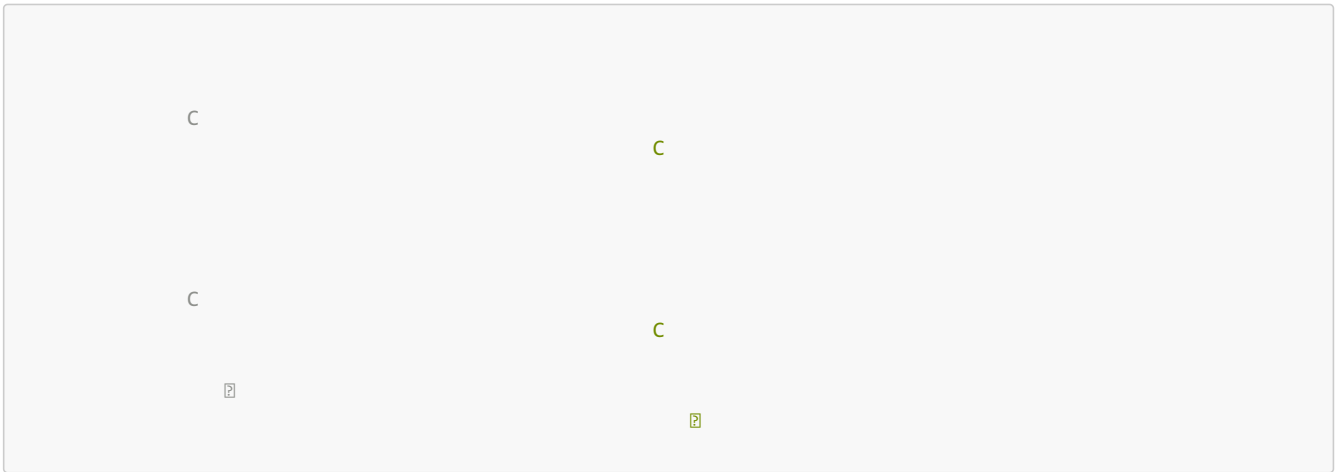




STEP 8: Build ChatOutput vertex

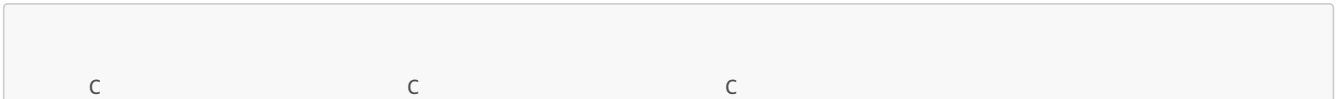


STEP 9: Events sent to frontend



7. Example 2: Calculator Tool Flow {#example-2-calculator-tool-flow}

The Flow



Calculator Component Code

C C C C
C C CA
C

C ?
A

Step-by-Step Execution

STEP 1: Request with math expression

? ?

C C C
C C C
C C

C
C

C
C

STEP 2: ChatInput processes input

C



STEP 3: Calculator receives the expression

C

C

C

C

C

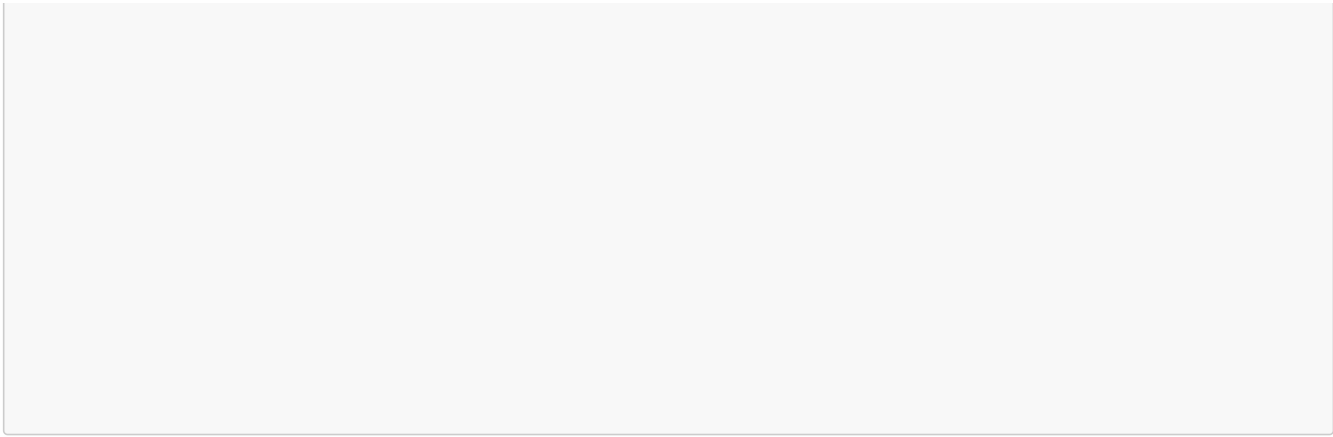
C

C

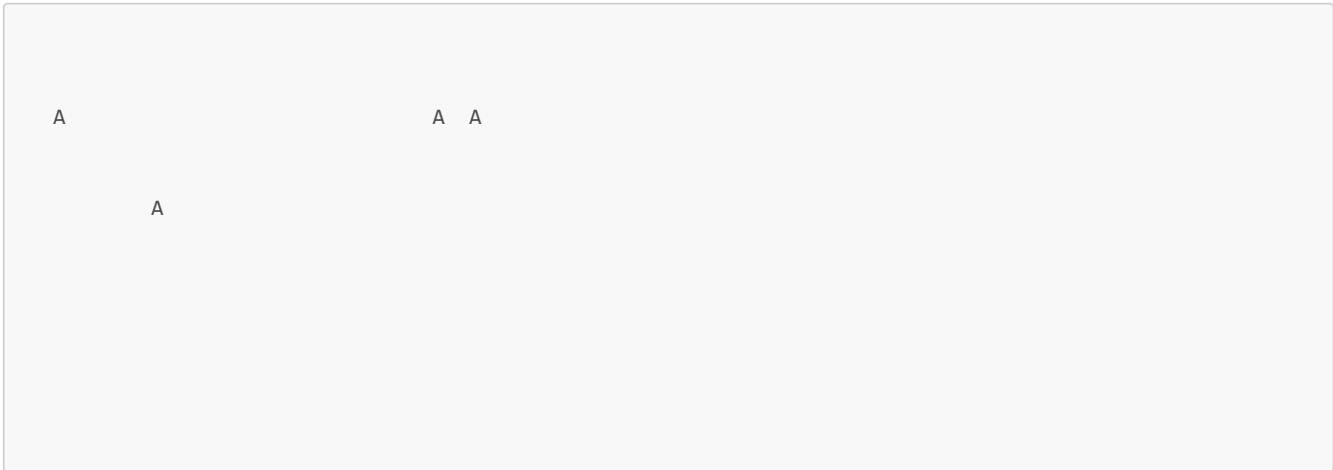
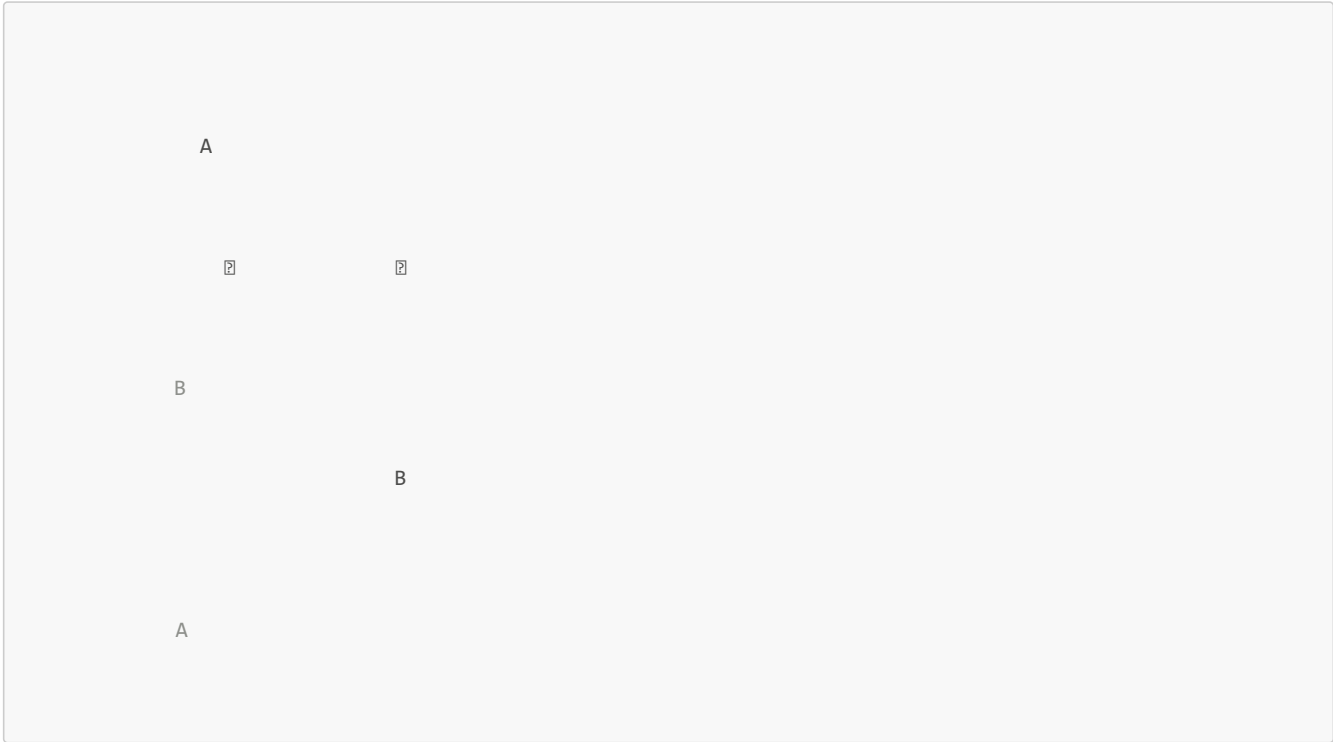
STEP 4: Calculator evaluates expression

A A

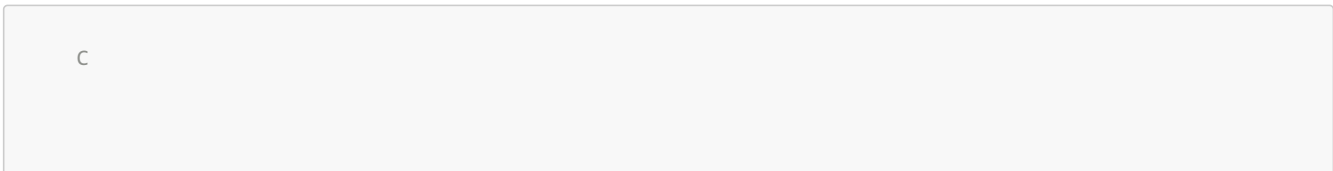
B
B
B
A

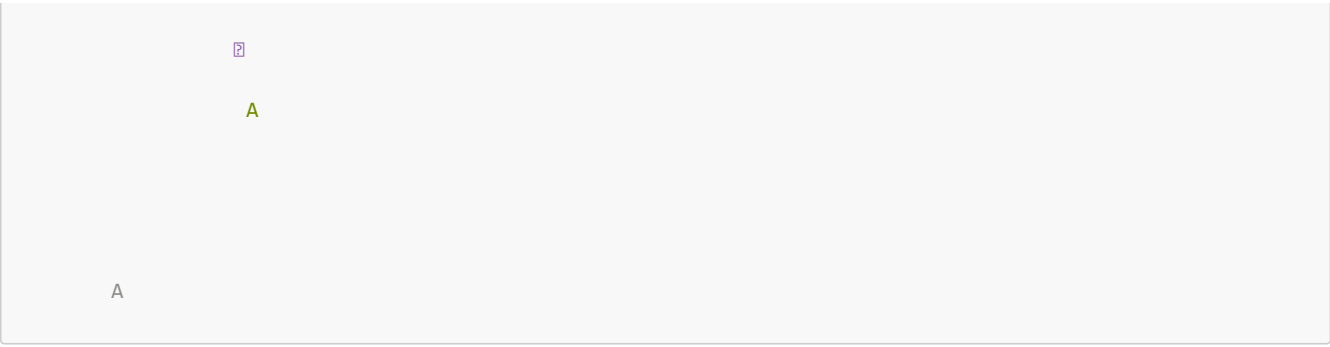


STEP 5: AST Evaluation Detail



STEP 6: ChatOutput displays result

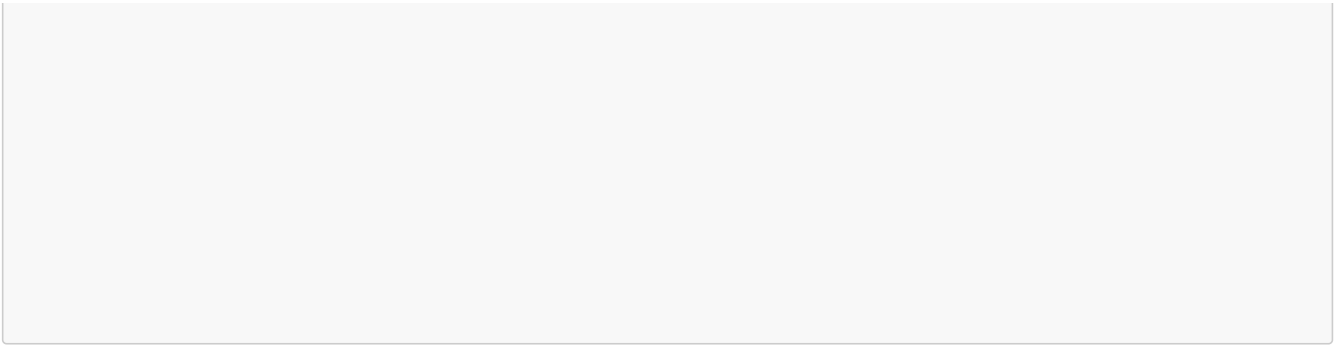




8. Complete Code Execution Trace {#complete-code-execution-trace}

Sequence Diagram: Full Flow Execution

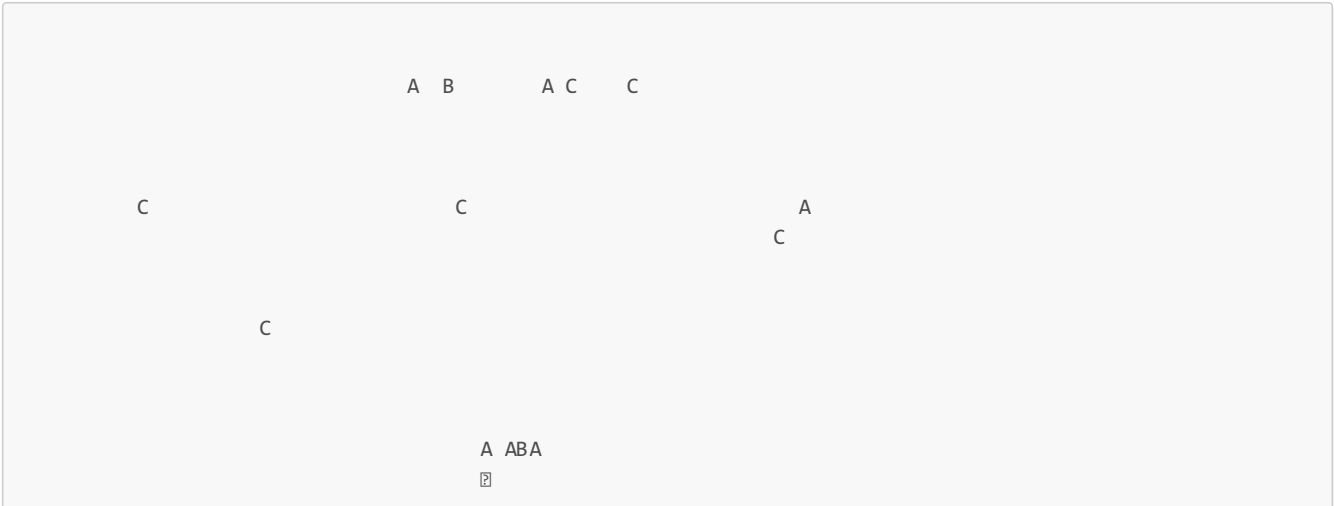


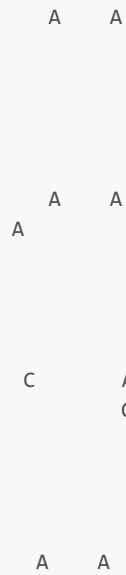


File-by-File Execution Order

Order	File	Function/Class	Input	Output
1		?	HTTP Request, flow_id, inputs	job_id
2		?	job_id, inputs, data	Event Queue
3		?	Event Manager	Events stream
4			session, flow_id	Graph/Adapter
5			JSON payload	LangGraphAdapter
6			nodes[], edges[]	Vertices built
7			-	first_layer[]
8			vertex_id	VertexBuildResponse
9	?		params	built_result
10			vertex	Component instance
11			code string	Class object
12		C	params	Instance
13	Component file (e.g.,)		self.input_value	Message
14	?		-	ResultData
15		Event emission	result	SSE to frontend

Summary: Complete Architecture





9. When is Each Folder Used? (UI vs Runtime) {#when-is-each-folder-used}

IMPORTANT CLARIFICATION

The **custom/code_parser/** folder is used for **BOTH** UI display **AND** runtime execution - NOT just for UI!

Complete Usage Table

Folder/File	Used for UI?	Used at Runtime?	Purpose
custom/code_parser/	YES	NO	Parse code to extract metadata for UI
custom/directory_reader/	YES	NO	Discover component files for menu
custom/attributes.py	YES	NO	Validate icons, names for UI
custom/eval.py	NO	YES	Convert code string → Python class
custom/custom_component/component.py	NO	YES	Base class - ALL components inherit this
custom/custom_component/custom_component.py	NO	YES	Flow control, tracing, storage
custom/custom_component/base_component.py	YES	YES	Code tree parsing + base functionality
custom/utlis.py	YES	YES	Build templates + runtime utilities
template/	YES	NO	Define UI field structure
template/frontend_node/	YES	NO	JSON for frontend rendering

Visual Timeline: When Each Part is Used

C

A

A

CA

A

A A

A

C

B

C

C

C

C

C

C

A

C

CA

A

A

C

C

?

?

C

C C

C

A

?

C

B

A

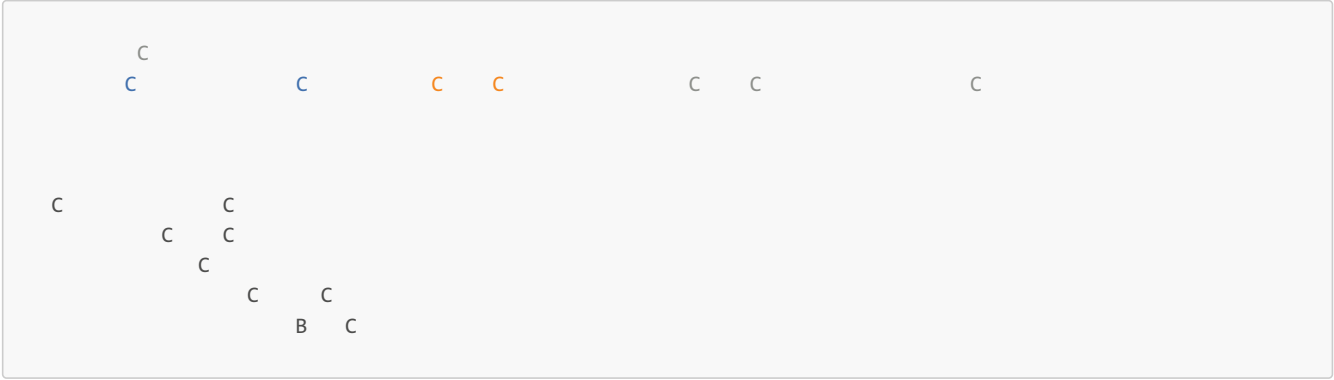
?

C



Why Component.py is CRITICAL at Runtime

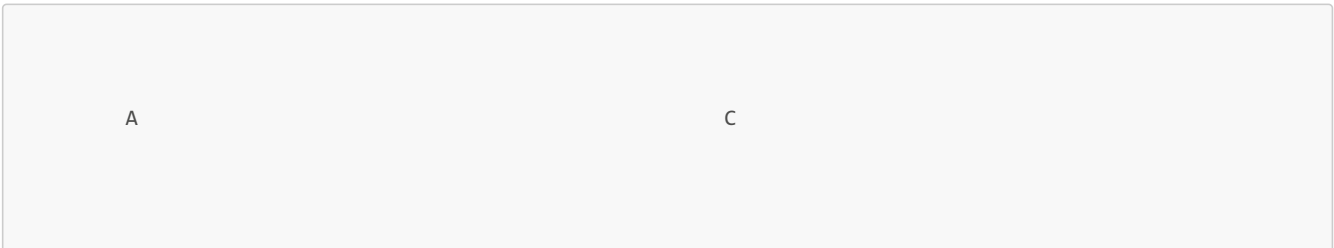
Every component you create inherits from C :

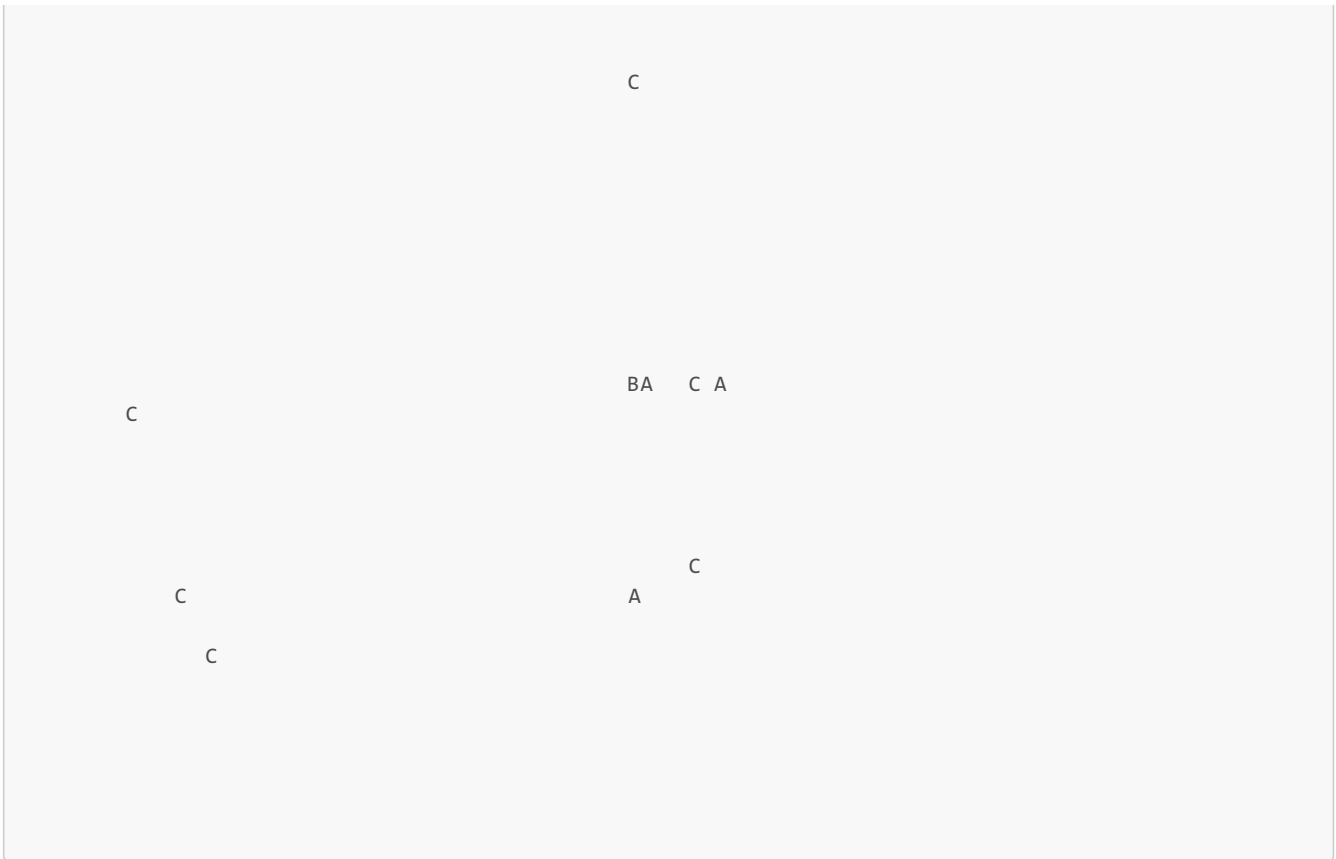


At runtime, your component USES these base class features:

Base Class Feature	What It Does at Runtime
C	Stores input values (e.g.,)
C	Makes work
C	Manages output values
C	Orchestrates method calls
C C	Sets status shown in UI
C C	Logs execution for debugging
C C	Access to flow graph
B C	Caches expensive operations

Summary: custom/ Folder is Used TWICE





10. Detailed Examples: UI Loading vs Runtime Execution {#detailed-examples-ui-vs-runtime}

Example A: Calculator Component - Complete Lifecycle

Let's trace a Calculator component from the moment the app starts to when it executes.

PHASE 1: Application Startup (UI Loading)



C

C

C
C

C

C
CA

C

A

C
B

C

C

C
C

C

C

CA

C

C

C

C

C

C

C

C

C

CA



PHASE 2: User Drags Calculator to Canvas (Still UI)



C

AC

C

C

A C C A A

C

C

C

C

C

C

C

B

C

C

?

C

?

?

?

?

A

AC

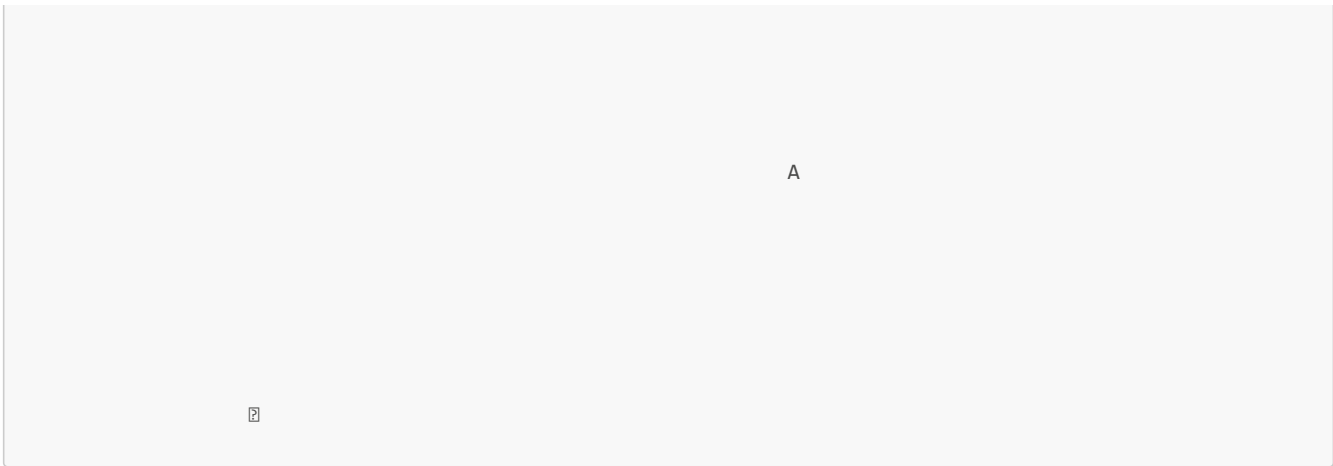
?

C

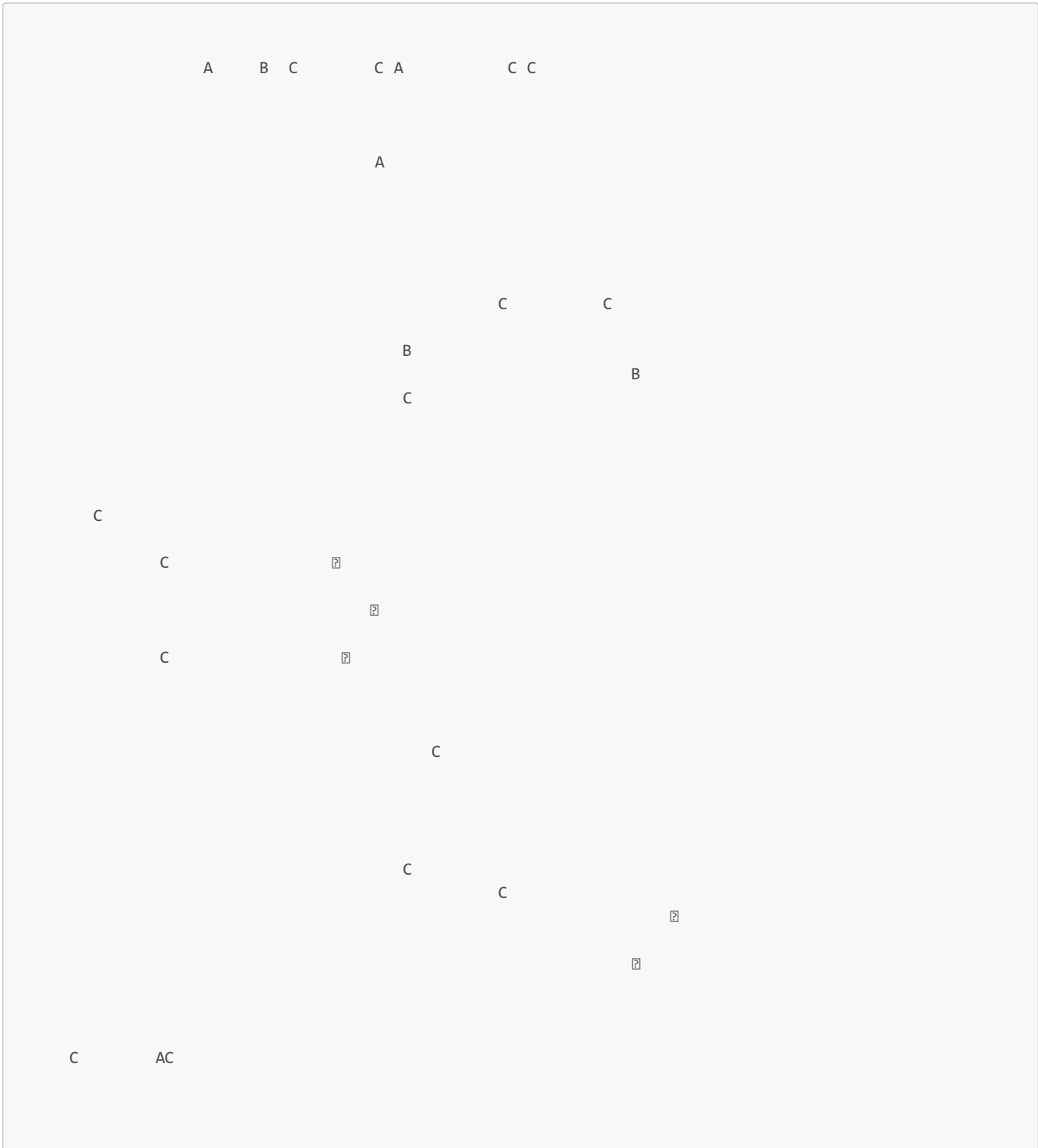
C

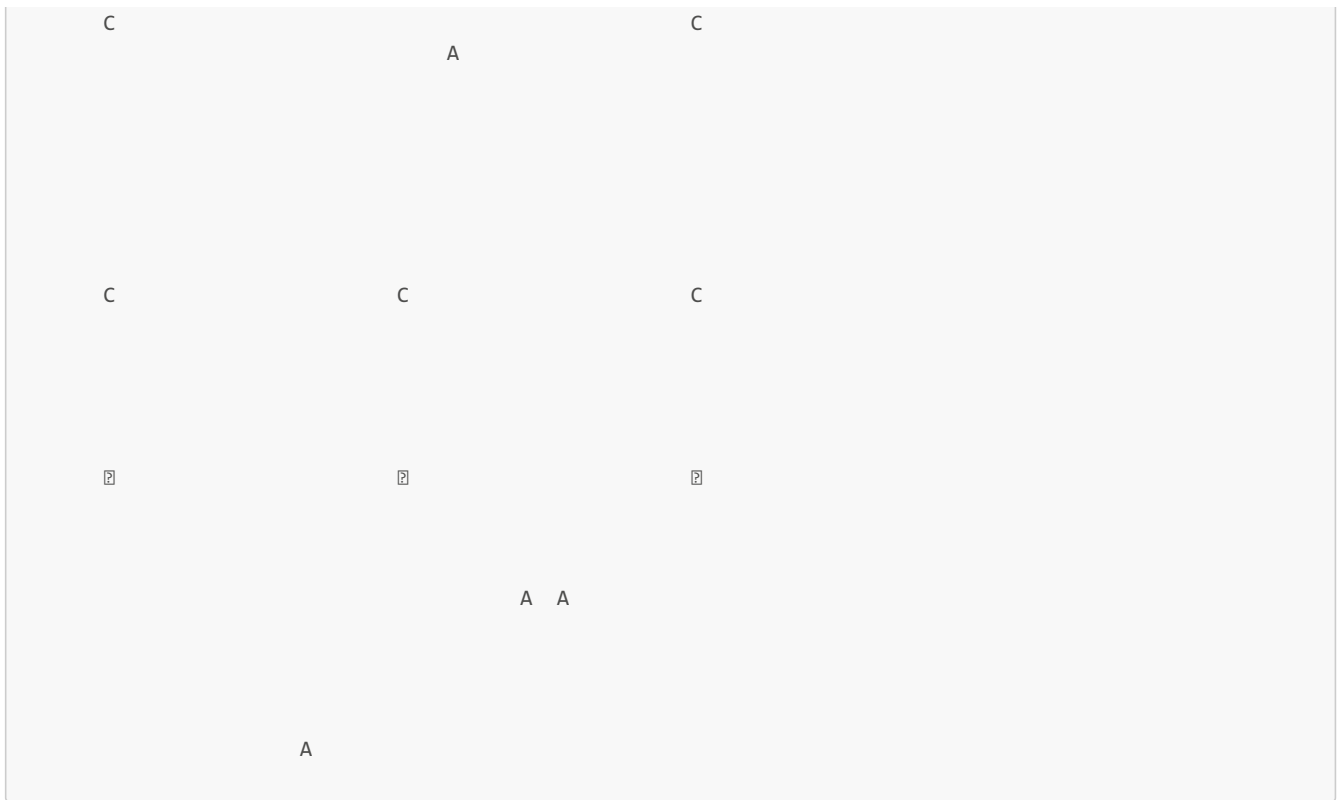
?

C



Example B: ChatInput → LLM → ChatOutput - Full Lifecycle





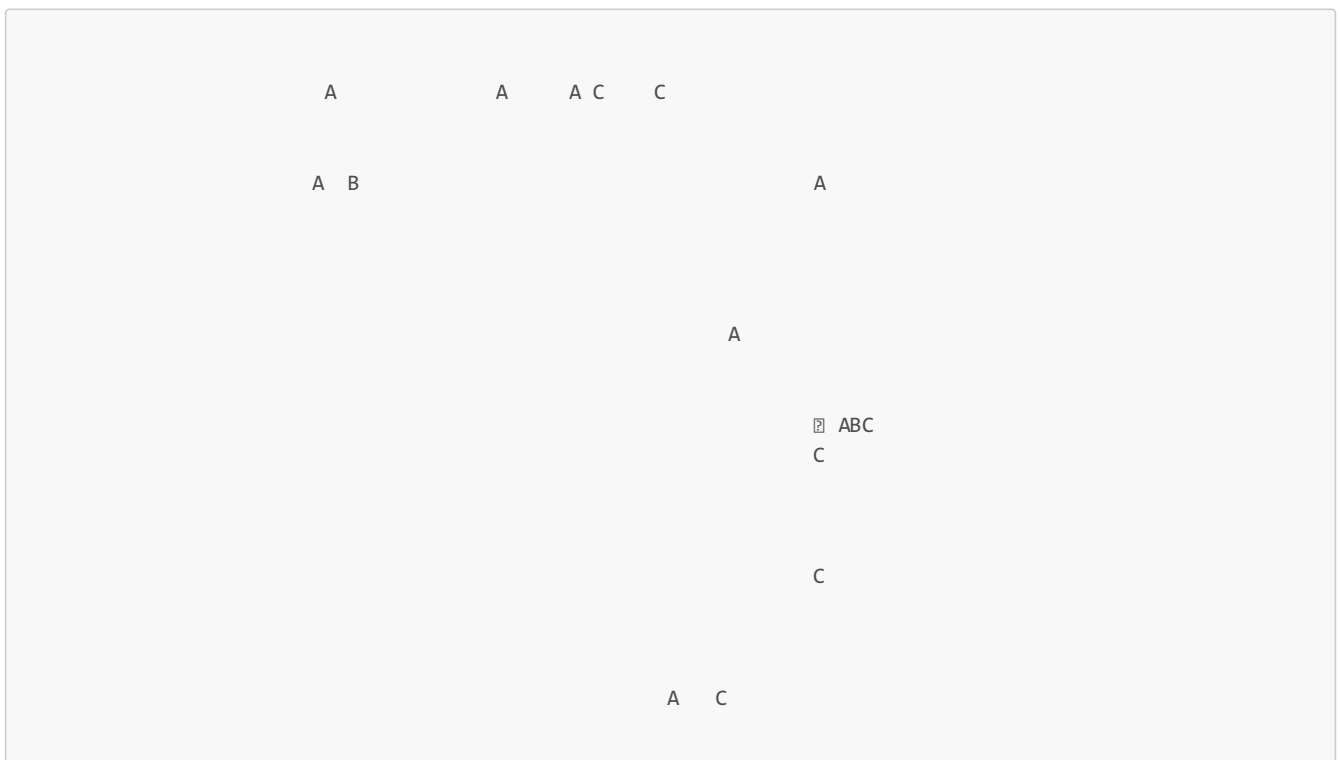
11. Tracing with Langfuse: Complete Integration {#tracing-with-langfuse}

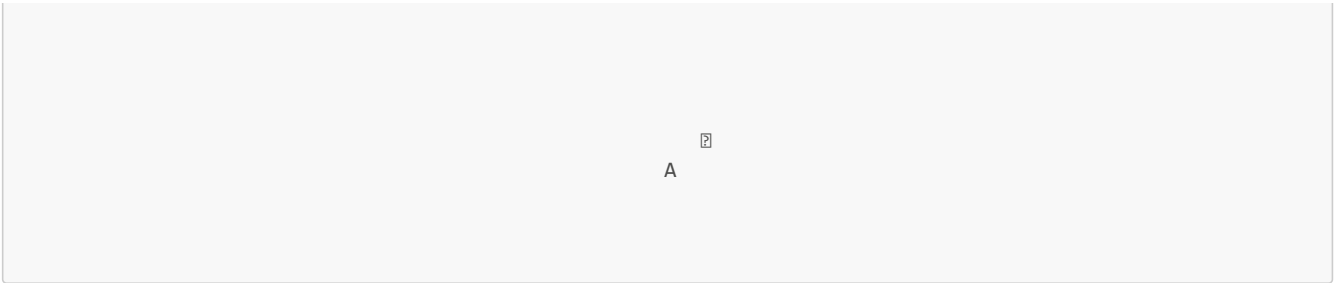
What is Langfuse?

Langfuse is an open-source LLM engineering platform for:

- **Tracing:** See every step of your LLM application
- **Debugging:** Find where errors occur
- **Analytics:** Track latency, cost, and quality
- **Evaluation:** Score your LLM outputs

How Langbuilder Integrates with Langfuse



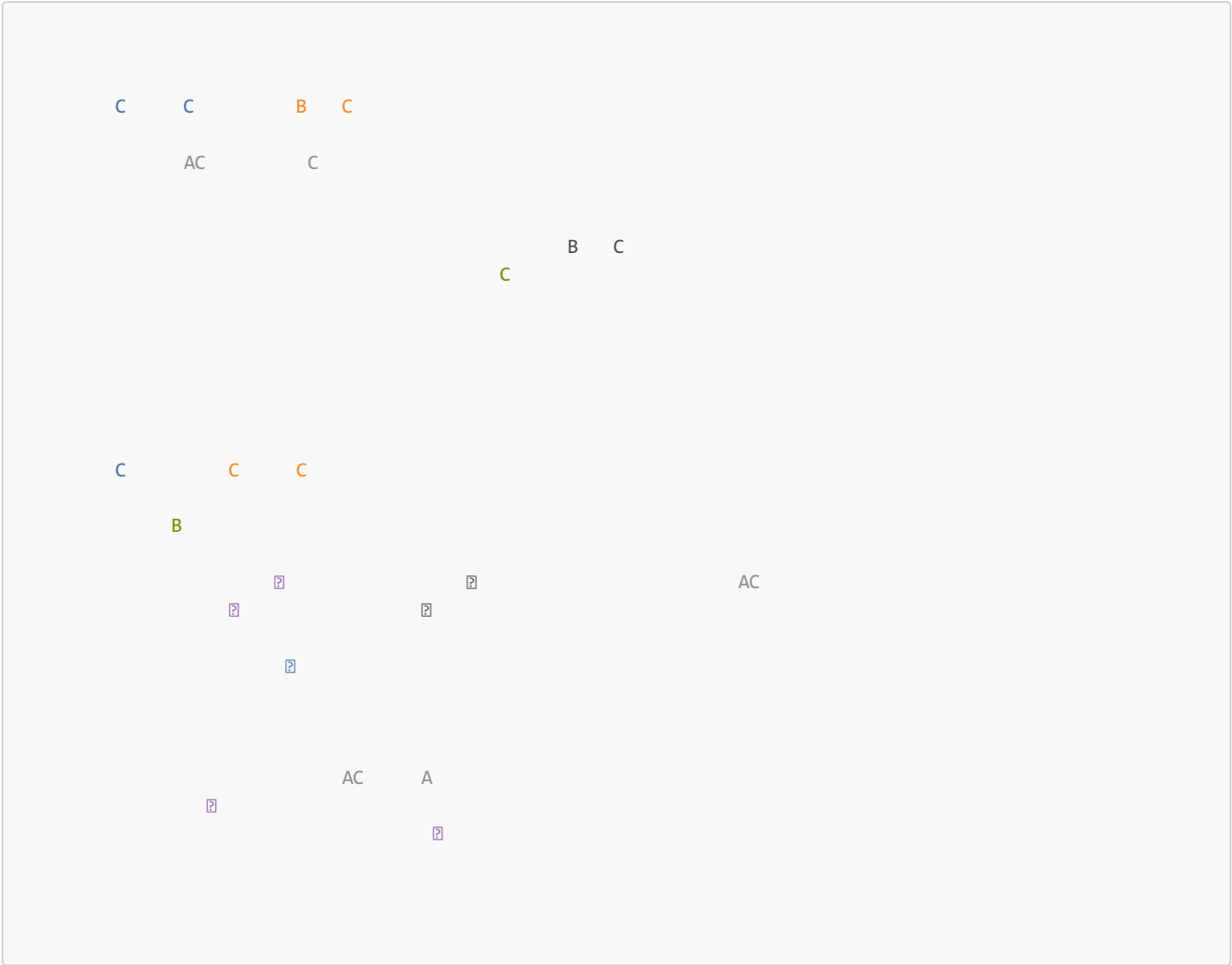


Key Files for Tracing

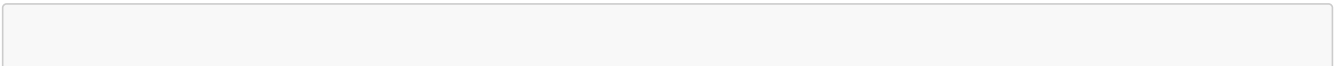
File	Purpose
	Main TracingService - orchestrates all tracers
	LangFuseTracer - Langfuse-specific implementation
	BaseTracer - Abstract interface for all tracers
	Creates TracingService instances
	Dependency injection for TracingService

How custom/ Folder Enables Tracing

The custom folder is **CRITICAL** for tracing because:



Complete Tracing Flow



AC

A C A

?

?

A

A

AC

?

?

?

?

?

C

?

?

AC

C

?

?

C

C

A

A

?

C

A

C

C

C

C

C

A

A

C

A

CA

BAC

A

A

C

AC

C

?

A

A

CA

BAC

A

C

A

C

C

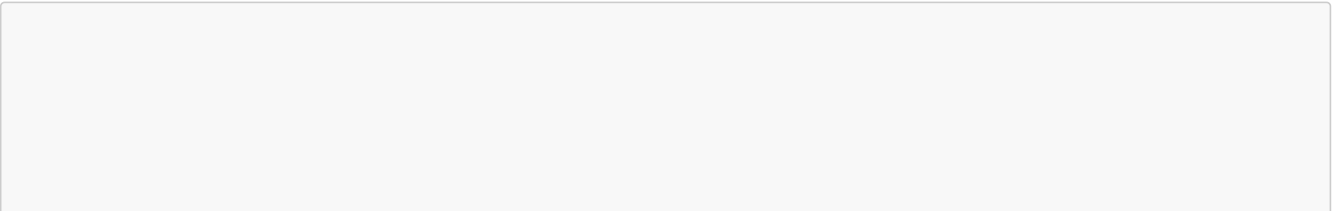
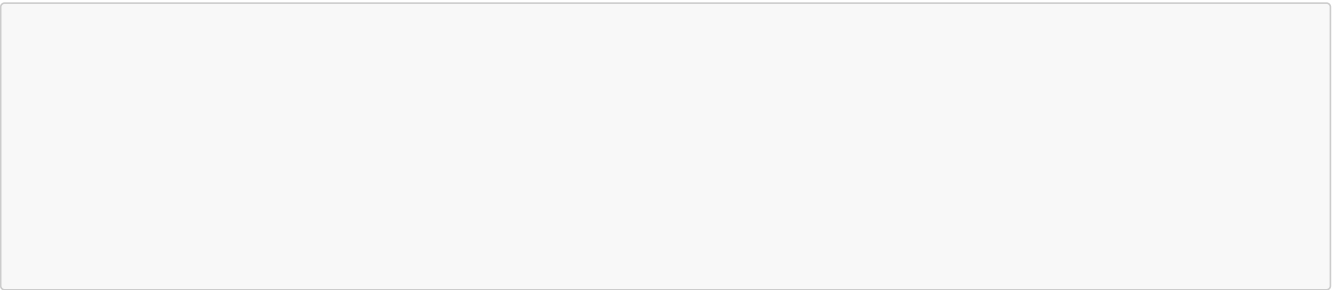
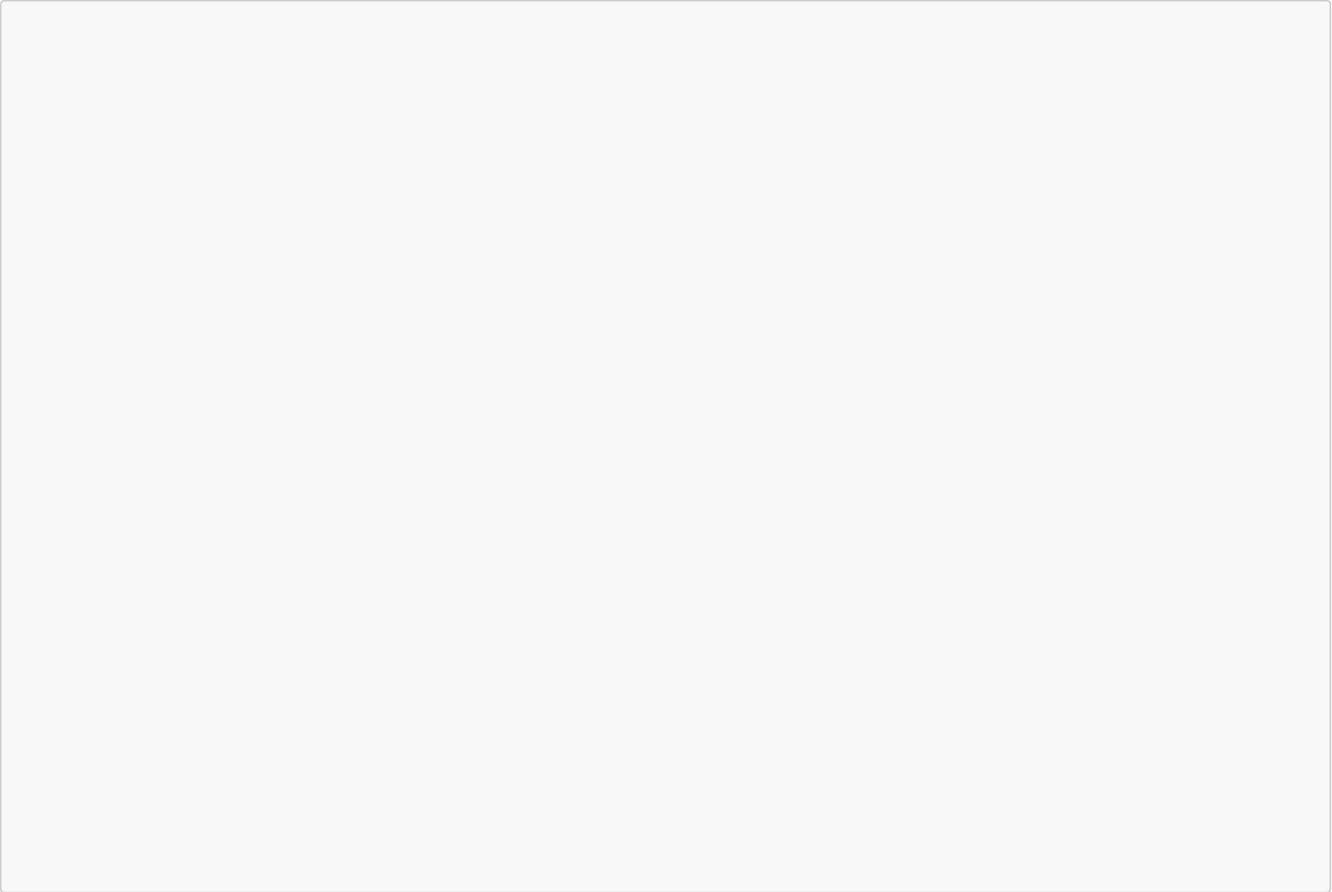
C

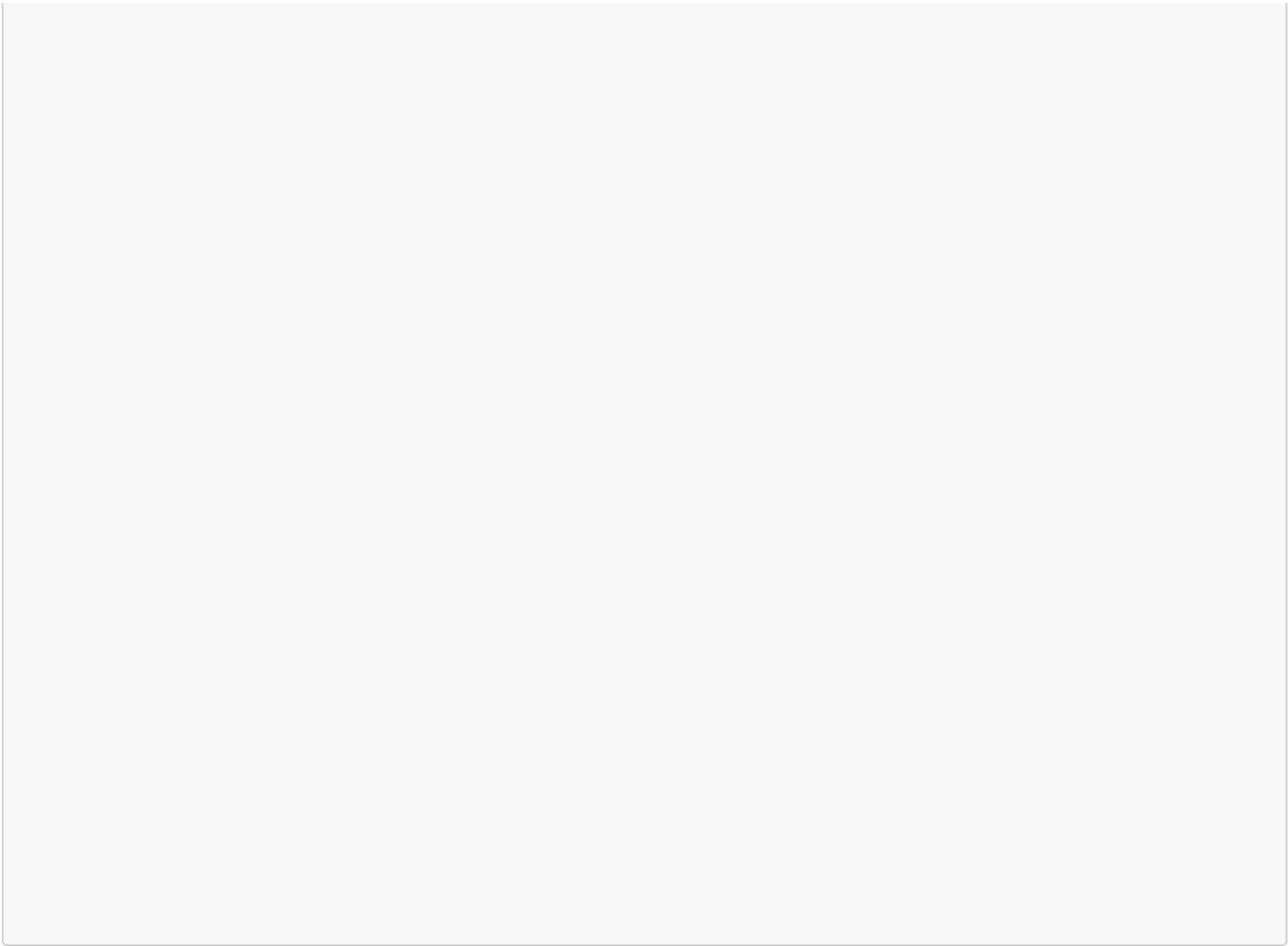
C

A

?

?





- **Calculator:** Shows UI loading (code_parser, attributes) vs Runtime (eval, component)
- **Chat Flow:** Shows complete lifecycle with tracing at every step